

Séance 3 — Calcul scientifique

Introduction à la programmation Python — IUT 1re année

Support de cours

Séance 3

Calcul scientifique

Modules et bibliothèques

Un **module** est un fichier `.py` contenant des fonctions réutilisables. Une **bibliothèque** (ou package) regroupe plusieurs modules.

- **Bibliothèque standard** : fournie avec Python (`math`, `random`, `csv`, `os`...)
- **Bibliothèques externes** : à installer soi-même (`numpy`, `matplotlib`...)

```
pip install numpy matplotlib
```

Importer un module

```
import math math.sqrt(16) # 4.0 from math import sqrt sqrt(16) # 4.0, sans préfixe
import numpy as np # alias conventionnel, très courant
```

Les fichiers : notions générales

- **Fichier texte** : contenu lisible (`.txt`, `.csv`, `.json`) — encodé en caractères (souvent UTF-8)
- **Fichier binaire** : données brutes non lisibles directement (`.png`, `.xlsx`)
- **Chemin** : localisation du fichier, relatif (`data/mesures.csv`) ou absolu (`/home/user/data/mesures.csv`)

Ouvrir un fichier avec `with`

```
with open("mesures.txt", "r", encoding="utf-8") as f: contenu = f.read()
```

Le bloc `with` garantit la **fermeture automatique** du fichier, même en cas d'erreur — c'est la méthode recommandée.

Modes d'ouverture courants : `"r"` lecture, `"w"` écriture (écrase), `"a"` ajout, `"x"` création.

Lire un fichier texte

```
with open("mesures.txt", "r", encoding="utf-8") as f: for ligne in f: # parcourt le fichier ligne par ligne print(ligne.strip()) # .strip() enlève le \n final
```

Lire un fichier CSV

```
import csv with open("mesures.csv", newline="", encoding="utf-8") as f: lecteur = csv.reader(f) next(lecteur) # ignore la ligne d'en-tête for ligne in lecteur: temps, temperature = ligne print(temps, temperature)
```

Écrire dans un fichier

```
with open("resultats.txt", "w", encoding="utf-8") as f: f.write(f"Moyenne : {moyenne:.2f}\n") f.write(f"Écart-type : {ecart_type:.2f}\n")
```

Qu'est-ce que NumPy ?

NumPy est LA bibliothèque de calcul scientifique en Python : elle fournit le type `ndarray` (tableau optimisé) et des opérations mathématiques rapides sur de gros volumes de données.

```
import numpy as np
```

Créer un `ndarray`

```
a = np.array([1, 2, 3, 4]) # tableau 1D à partir d'une liste b = np.zeros((3, 4)) # tableau 3x4 rempli de zéros c = np.arange(0, 10, 2) # [0, 2, 4, 6, 8] d = np.linspace(0, 1, 5) # 5 valeurs régulières entre 0 et 1
```

Dimensions et taille

```
a = np.array([[1, 2, 3], [4, 5, 6]]) a.shape # (2, 3) : 2 lignes, 3 colonnes a.ndim # 2 (nombre de dimensions) a.size # 6 (nombre total d'éléments) a.dtype # type des éléments (int64, float64...)
```

Accéder aux valeurs

```
mesures = np.array([[12.1, 12.5, 12.9], [20.0, 20.2, 19.8]]) mesures[0, 1] # 12.5  
: ligne 0, colonne 1 mesures[1, :] # toute la ligne 1 mesures[:, 0] # toute la  
colonne 0
```

Opérations vectorisées

NumPy applique les opérations à **tout le tableau** sans écrire de boucle explicite — c'est plus rapide et plus lisible.

```
temperatures_c = np.array([20.0, 21.5, 19.8]) temperatures_k = temperatures_c +  
273.15 # appliqué à chaque élément double = temperatures_c * 2
```

Statistiques avec NumPy

```
mesures = np.array([12.1, 12.5, 12.9, 13.0]) mesures.mean() # moyenne  
mesures.std() # écart-type mesures.min() # minimum mesures.max() # maximum  
mesures.sum() # somme # Sur un tableau 2D, préciser l'axe : tableau =  
np.array([[1, 2], [3, 4]]) tableau.mean(axis=0) # moyenne par colonne  
tableau.mean(axis=1) # moyenne par ligne
```

À vous de jouer

Direction les exercices de la séance 3 →

Exercices

Exercices — Séance 3 : Calcul scientifique

Ex. 1 — Module `math` et `random` Utiliser `math` pour calculer racine carrée, puissance, et `random` pour générer 10 mesures aléatoires simulées entre 10 et 20.

Ex. 2 — Lecture d'un fichier texte Lire un fichier `notes.txt` (une valeur par ligne) et compter le nombre de lignes ainsi que la somme des valeurs.

Ex. 3 — Lecture d'un CSV de mesures Charger un fichier `mesures.csv` (colonnes temps, température) dans deux listes Python, puis calculer la moyenne "à la main".

Ex. 4 — Écriture d'un rapport Écrire les résultats de l'Ex. 3 (moyenne, min, max) dans un fichier `rapport.txt`, avec `with open(..., "w")`.

Ex. 5 — Premier tableau NumPy Créer un `ndarray` à partir des données de l'Ex. 3, afficher `shape`, `dtype`, puis convertir les températures de °C en K en une seule opération vectorisée.

Ex. 6 — Statistiques et tableau 2D Construire un tableau NumPy 2D (3 capteurs × 5 instants, valeurs de votre choix) et calculer la moyenne par capteur (`axis=1`) et par instant (`axis=0`).